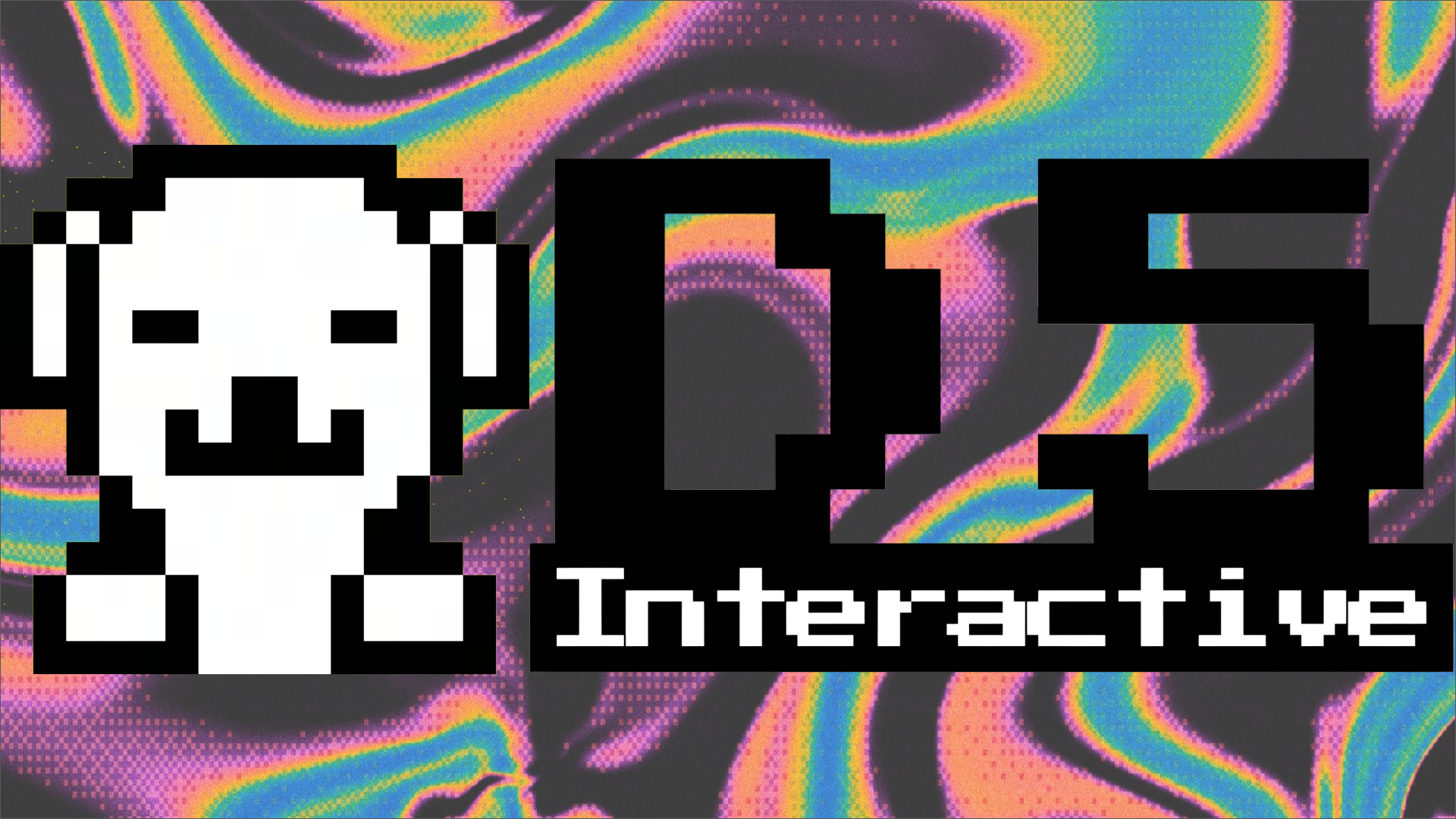

Interactive

D5-INTERACTIVE & THE CYBER JEDIS PROUDLY PRESENT...

# WORK TOGETHER OR DIE TOGETHER

CYBER JEDIS

D5 interactive



# STARSEC

LIVE PRE-ALPHA SHOWCASE - ROWDY CYBERCON NOV 7TH

d5i.store

D5-INTERACTIVE & THE CYBER JEDIS PROUDLY PRESENT...

WORK TOGETHER OR DIE TOGETHER

CYBER JEDIS

D5 interactive

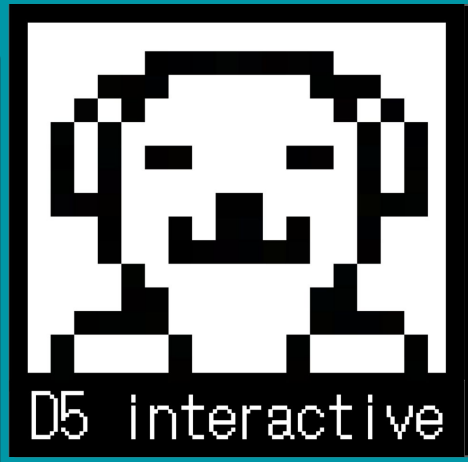


STARSEC

LIVE PRE-ALPHA SHOWCASE - ROWDY CYBERCON NOV 7TH




# Intro to Docker






The Cyber Jedis & D5i

OCT 3RD 2026 - RowdyHacks

Presented by: Walker Caskey

What is docker?
Why should i use it?
Is it necessary?
What can i do with it?



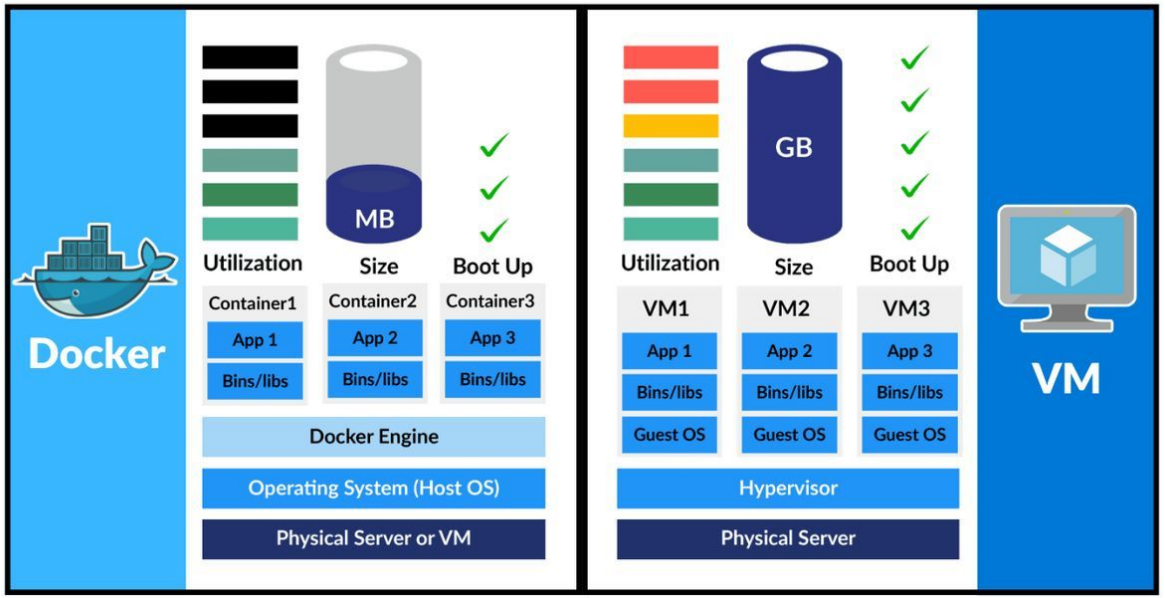
# History of Docker – What came before

A Virtual Machine is a full, software-emulated computer running inside your real computer. If you've used VirtualBox or VMware to run Kali Linux inside your Windows machine, you've used a VM. The VM has its own simulated CPU, RAM, storage, and network card. It runs a complete, separate operating system (called a "guest OS") entirely isolated from your main system (the "host OS").

This works, but it's heavy. A typical Kali Linux VM image is 3–12 GB. It needs gigabytes of RAM allocated to it at all times. It can take 1–3 minutes to boot. Running three VMs simultaneously will bring most laptops to their knees.

# Differences

Containers take a fundamentally different approach. Instead of emulating an entire computer and running a separate OS, containers share the host machine's OS kernel;
the core of the operating system; and only package the application and its specific dependencies on top of that. This makes them dramatically smaller and faster.



# Container Characteristics

**Isolated Process:** From the container's perspective, it thinks it's the only thing running on the machine. It has its own file system, network interfaces, and process list.

**Immutable by default:** Any changes you make inside a running container (installing packages, editing files) are thrown away when the container stops, unless you explicitly save them.

**Defined by an Image:** Every container is born from a Docker Image — a read-only template that describes the container's contents. Think of an *Image* as a *blueprint*, and the Container as the finished output / product.

**Layered:** Images are built in layers. If two images both use the same Ubuntu as a base, they share that layer, rather than duplicating it.

# Internal Processes

Docker uses a client-server architecture. There are three main components:

**Docker CLI (Client):** This is the terminal commands you type — docker run, docker ps, etc. It sends your commands to the Daemon.

**Docker Daemon (dockerd):** The background service that does all the actual work — building images, starting containers, managing networking and storage.

**Docker Registry:** A repository where Docker Images are stored and shared. Docker Hub (hub.docker.com) is the public one — it hosts thousands of pre-built images for everything from Nmap to full Kali Linux environments.

# What Actually Isolates Containers?

Docker uses ***TWO*** core Linux kernel features that have existed for years:

- Namespaces
- Control Groups (cgroups)

# Namespaces: The Illusion of Being Alone

A **namespace** wraps a system resource in an abstraction layer so that processes inside the namespace think they have their own isolated instance of that resource. Docker uses ***6types*** of namespaces:

| Namespace | What It Isolates | Example in Practice |
|---|---|---|
| pid | Process IDs | Container thinks its main process is PID 1 (like a fresh OS boot) |
| net | Network interfaces & routing | Container gets its own virtual network card and IP address |
| mnt | File system mount points | Container sees its own root filesystem, not the host's |
| uts | Hostname and domain name | Container can have its own hostname (e.g., "nmap-scanner") |
| ipc | Inter-process communication | Containers can't share memory with host processes |
| user | User and group IDs | Root inside a container ≠ root on the host machine |

# Control Groups: Resource Limits

While namespaces provide isolation, cgroups control how much of the host's resources a container can use.

They prevent a single runaway container from eating all your CPU or RAM and crashing everything else.

You can set hard limits like "this container gets at most 512 MB of RAM and 1 CPU core."

On the STARSEC backend for example, we limit each container to ~40MB RAM since there can be (potentially) multiple tens of containers in various different networks... And the server runs on a RPI 5.

# Union File System (Layered Storage)

Docker images are stored as stacked, read-only layers.

When you run a container, Docker adds a thin, writable layer on top.

Any files you create or modify go into that writable layer — the base image layers beneath are untouched.

When the container is deleted, that writable layer is discarded.

Layer 1: Base image (ubuntu, debian trixie, alpine, etcetc)

Layer2: Read only (Running basic updates via apt update && upgrade, installing deps & tools such as nmap, etc)

Layer3: NMAP configuration (config layer)

Then you would start the container, and it would delete anything outside a volume; fresh.

Because containers share the host kernel, a kernel-level vulnerability could potentially allow a container escape; where malicious code inside a container breaks out to the host.

This is rare and requires sophisticated exploits, but it's why you should never run untrusted container images as root.

VMs provide stronger isolation for truly hostile workloads.

# How Docker Works on Linux, Windows & macOS

Docker containers require a Linux kernel. Since containers share the host's kernel, what happens on non-Linux machines?

On Linux, Docker runs natively.

The Docker daemon talks directly to the Linux kernel's namespaces and cgroups. There is zero overhead — containers are essentially just isolated Linux processes.

Performance is as close to bare-metal as possible.

This is why most production servers and security labs run Docker on Linux.

# macOS

macOS uses its own kernel (Darwin/XNU), which is not Linux.

So when you install Docker Desktop for Mac, it silently creates a small, lightweight Linux VM (using Apple's Hypervisor framework) that runs in the background.

Your Docker containers actually run inside that hidden Linux VM. This adds a small layer of overhead, but Docker Desktop manages it transparently, so you'll never notice it in day-2–day use unless you have a particularly low-spec machine.

All your docker commands still work the same way from your macOS terminal.

# Windows

Windows also doesn't run a Linux kernel natively (unless you're using WSL2). Docker Desktop for Windows can use one of two backends:

**WSL2 Backend (Recommended):** Windows Subsystem for Linux 2 provides a real Linux kernel running alongside Windows. Docker integrates with it directly, giving near-native Linux performance. This is the default and preferred option.

**Hyper-V Backend (Legacy):** Uses Microsoft's Hyper-V hypervisor to create a Linux VM. Heavier than WSL2.

On both macOS and Windows, Docker Desktop provides a GUI application that lets you manage containers visually, though most professionals use the CLI. Portainer and other adjacent GUI dashboards exist for linux users.

# What About Windows Containers?

Docker can run Windows containers that use the Windows kernel instead.

These are rare and used primarily for enterprise Windows apps.

In cybersecurity, you'll almost exclusively work with Linux containers — which is what this guide covers.

# INSTALLING & SETUP

Now we'll show and help set up docker for your machine!

# On macOS / Windows

Go to:

***https://www.docker.com/get-started/***

**Click on your OS's download link**

**Wait for download to finish, run the installer.**

**It can take up to 5 minutes on some machines.**

# On Debian Linux

I have a small script / set of CMDS that will remove any old or outdated editions, and reinstall / install fresh.

Go to:

**https://shorturl.at/8clNd**

Then, copy and paste it into your terminal and press enter;

or save it as a file and run ‘bash <filename>.sh’.

# Test your installation

Run”docker run hello-world”.

Docker will pull a tiny test image from Docker Hub and run it.

If you see "Hello from Docker!" printed in your terminal, everything is working correctly.

Essential docker commands can be found in the docker-workshop repo.

https://github.com/dsient/docker-workshops/tree/main

# Docker Images & Dockerfiles

A Docker Image is a read-only template that defines what a container will look like when it runs. You can use pre-built images from Docker Hub, or build your own custom images using a Dockerfile.

A Dockerfile is a plain text file with a set of instructions that Docker follows to build an image. Think of it like a recipe card; "Start with Ubuntu, install these packages, copy these files, and when you run this container, start this program."

Simple enough once you get the hang of it.

# Lets make a custom image!

Got docker installed and ready?

Make a new folder and open nano or notepad!

Note that a Dockerfile has no extension.

The name should be “Dockerfile”, exactly, otherwise it wont be able to find it.

(This is different from docker compose files, which are .yaml)

# Now for something more exciting…

**# Create an isolated network for your lab**

docker network create --subnet=172.20.0.0/16 lab

**# Start a vulnerable target on the lab network**

docker run -d --name target --network lab vulnerables/web-dvwa

**# Start Kali attacker container on the same network**

docker run -it --name attacker --network lab kalilinux/kali-rolling bash

**# Inside attacker container — target is reachable by container name!**

root@attacker:~# nmap -sV target

root@attacker:~# curl http://target/

**# Clean up entire lab**

docker stop target attacker ; docker rm target attacker ; docker network rm lab

## Tip — Persisting Scan Results

Always mount a volume to save your work:

```
docker run --rm -v $(pwd)/results:/output ....
```

This maps a results/ folder in your current directory to /output inside the container. Anything saved there survives when the container is deleted.

***Practice your Docker skills in context:***

HackTheBox and TryHackMe both provide Docker-based lab environments.
VulnHub offers downloadable vulnerable VMs you can attack from a Kali container.
PentesterLab has Docker-specific security challenges.

# More Essential Security Tool Images

| TOOL | DOCKER IMAGE | PURPOSE |
| --- | --- | --- |
| Burp Suite CE | securecodebox/burp | Web app proxy & vulnerability scanner |
| SQLMap | paoloo/sqlmap | Automated SQL injection detection |
| Gobuster | kodisha/gobuster | Directory/subdomain brute-forcing |
| Nikto | securecodebox/nikto | Web server vulnerability scanner |
| John the Ripper | phocean/john | Password cracking |
| Hashcat | dizcza/docker-hashcat | GPU-accelerated password cracking |
| OpenVAS | securecompliance/gvm | Full vulnerability management suite |
| WPScan | wpscanteam/wpscan | WordPress security scanner |
| Juice Shop | bkimminich/juice-shop | Vulnerable Node.js app for training |
| HackTheBox PWN | Various CTF images | Capture the Flag challenge environments |

# Docker Compose — Orchestrating Multi-Container Labs

Running one container at a time is powerful. Running an entire attack lab with an attacker, target, database, logging and more with a single command is transcendent.

So far, every Docker command we've run manages one container at a time. This works fine for simple tools, but real-world applications (and realistic cybersecurity labs) involve multiple components talking to each other.

# Consider:

A penetration testing lab that needs:

- A vulnerable web application (DVWA) as the target
- A MySQL database that the web app connects to
- A Kali Linux container as the attacker
- An ELK Stack (Elasticsearch, Logstash, Kibana) to collect and visualize logs

Managing four containers manually — starting them in the right order, connecting them to the same network, setting environment variables, mounting volumes quickly becomes the bane of your existence.

You'd need to run four docker run commands with dozens of flags, and remember to tear them all down afterward.

## ***Docker Compose solves this.***

It lets you define your entire multi-container environment in a single YAML file called docker-compose.yml, then start or stop the whole thing with *one command*.

Single File Definition

Your entire lab lives in one docker-compose.yml. Version control it, share it with teammates.

One Command Deploy

‘docker compose up’ starts everything.
‘docker compose down’ tears it all down.

Auto Networking

All services in a Compose file automatically get a shared private network.
They can reach each other by service name

Dependency Ordering

Declare that your web app depends on a database, and Compose starts the database first.

# Docker Compose v1 vs v2

Older tutorials use ‘docker-compose’ (with hyphen) ; that was Compose v1, written in Python and installed separately.

Modern Docker ships with Compose v2 built in, accessed as docker compose (no hyphen, as a Docker plugin).

***Always use ‘docker compose’ going forward.***

# YAML Basics

The docker-compose.yml file is written in YAML (YAML Ain't Markup Language) .

Its a human-readable data format that uses indentation to show structure.

Therefore, Indentation matters enormously [use spaces, never tabs]

```yaml
# A key-value pair
name: my-lab

# A list (each item starts with a dash)
ports:
  - "8080:80"
  - "443:443"

# A nested structure (indented 2 spaces)
environment:
  MYSQL_USER: admin
  MYSQL_PASSWORD: secret123

# A boolean
tty: true
```

# Anatomy of docker-compose.yml

https://raw.githubusercontent.com/dsient/docker-workshops/refs/heads/main/docker-compose.yml

# Service Discovery

This is one of the most powerful features of Docker Compose.

When multiple services are on the same network, they can reach each other using the service name as a hostname.

Docker's internal DNS resolves service names automatically.

```
# In docker-compose.yml you have:
services:
  webapp:
    image: my-app
  database:
    image: mysql:8.0

# Inside the 'webapp' container, you connect to MySQL like this:
# Host: database    (not localhost, not an IP - the SERVICE NAME)
# Port: 3306

# Similarly, inside 'attacker' container:
# curl http://webapp:80      ← reaches the web app by service name
# nmap -sV database          ← scans the database container
```

# Docker Lab Ideas

## Lab 1 — DVWA Full Stack + Kali Attacker

A simple but realistic web app pentesting lab. DVWA with its own MySQL database, all on an isolated network, with Kali ready to attack.

## Lab 2 — OWASP Juice Shop + Network Monitoring

OWASP Juice Shop is a more modern, realistic vulnerable Node.js app with 100+ intentional vulnerabilities. Pair it with a traffic capture sidecar.

## Lab 3 — ELK Stack + Vulnerable App (Security Monitoring Lab)

A blue-team focused lab: Juice Shop generates logs, Filebeat ships them to an ELK stack, and you analyze attack patterns in Kibana.

## Lab 4 — Portable CTF Challenge Box

Package a custom CTF challenge so any teammate can run it with one command — no setup instructions needed.

# More & Advanced Topics

**Dockerfile Best Practices**

Learn multi-stage builds to create minimal production images. Smaller images mean smaller attack surface.

**Kubernetes / QEMU**

The next level after Compose — container orchestration at scale. Used in enterprise security operations centers.

**Container Forensics**

Learn to analyze container layers with docker history and dive to find hidden data in images.

**Container Escapes**

Study privileged container escapes and container breakout techniques — and how to prevent them.

**Rootless Docker**

Run Docker entirely without root privileges for hardened environments. Defense in depth principle.

**CI/CD Pipelines**

Integrate Docker with GitHub Actions to automatically build, scan, and deploy security tools on every commit.